COP CarGo App

Summary of Changes

Period covered: Sunday, October 18 to Friday, October 23, 2026
Prepared by: Frances Emmanuel Callado
Branch: dev-emmanuel

Overview

This week's work covered three areas:

- **Events and Services:** leaders can now choose the event or service their trip is for, and the list adjusts to the campus they pick.
- **Accountant user:** a first version of the new Accountant pages, including remarks on trip entries.
- **Fixes:** login redirection, the Accounting export link, and the Campus dropdown.

Part 2 of this document shows the code changes for each file.

1. Events and Services

What changed for users

- The trip entry form in the Leader's Portal now has an **Event / Service** dropdown.
- When a leader picks a campus, the dropdown reloads and shows only the events for that campus, plus events open to all campuses.
- Only events that are currently running are shown. Events without dates are always shown.
- “Special Event” entries are listed first, and the first event in the list is selected automatically.
- The dropdown starts with the prompt “Select Event or Service”.
- “Special Event” no longer appears as a choice in the Campus dropdown.

What changed behind the scenes

- Events now have a **Campus** (optional; leave it blank to show the event on every campus) and a **Priority** number. The start date, end date and description are now optional, and the description can hold longer text.
- Added a new lookup that the form uses to load events for a campus (`/api/get-events/`).
- Events can be managed in the Admin page, including the new Campus field.

Date	Commit
Oct 18	Added event prioritization on Events model
Oct 18	Added filtering and event-focusing on Event dropdown
Oct 18	Added campus field in the Event model or table
Oct 19	Removed the Special Service in the list of Campus
Oct 19	Updated Event or Service dropdown
Oct 19	Fixed field type for Event Description
Oct 19	Made fields in Events model nullable
Oct 23	Implemented the Event or Service field in the Leader's Portal

2. Accountant User and Remarks

What changed for users

- First version of the new Accountant dashboard, with a separate page that shows the full details of each trip entry.
- Accountants (and other roles) can now add remarks to a trip entry. Every remark is saved with who wrote it and when, so there is a full history instead of a single remarks field.
- A redesigned Accountant page built with Vue / Quasar has been added for review. The current page is still the one in use.

Date	Commit
Oct 19	Archive / for review -- revamped Account page using Vue / Quasar
Oct 19	Initial implementation of the Accountant user
Oct 19	Added remarks

3. Fixes and Setup

- **Fixed redirection:** after logging in, users go to the correct page for their role. The old redirection code was simplified.
- **Fixed the Export link** on the Accounting page.
- Added **default data for the role landing pages** (Pastor, Finance, Leader, Accountant), so they can be restored quickly on a new or reset database.

Date	Commit
Oct 19	Fixed redirection
Oct 19	Fixed export url in Accounting
Oct 20	Added default backup data for AuthRoutes

Files Changed

Lines added (+) and removed (−) in each file, not counting changes that only add or remove spaces or blank lines.

File	What changed	+ / −
`backup/auth_routes.json`	Default landing page for each role **NEW**	+42 / −0
`cashier/admin.py`	Event list in Admin shows the campus	+7 / −5
`cashier/forms.py`	Trip form uses the campus-based event list; “Special Event” removed from Campus choices	+22 / −12
`cashier/models.py`	Event: added campus and priority; dates and description are now optional	+18 / −7
`cashier/old_forms.py`	Removed an unused setting on the Campus field	+0 / −2
`cashier/remarks_urls.py`	Address for submitting a remark **NEW**	+7 / −0
`cashier/remarks_views.py`	Saves and lists remarks **NEW**	+69 / −0
`cashier/urls.py`	New addresses for event lookup and remarks	+7 / −1
`cashier/utils.py`	Picks which events a campus should see **NEW**	+31 / −0
`cashier/views.py`	New event lookup for the form; login redirection fixed; Accounting page uses the new dashboard	+23 / −27
`templates/Accountant/base.html`	Page layout for Accountant pages **NEW**	+33 / −0
`templates/Accountant/entry-detail.html`	Full details of a trip entry **NEW**	+107 / −0
`templates/Accountant/index-2.html`	Redesigned Accountant dashboard **NEW**	+411 / −0

File	What changed	+ / −
`templates/Accountant/index.html`	Fixed the Export link	+1 / −1
`templates/Accountant/remarks.html`	Remarks history section **NEW**	+30 / −0
`templates/Accountant/styles.html`	Styling for Accountant pages **NEW**	+352 / −0
`templates/cargo_form.html`	Event dropdown reloads when the campus changes	+14 / −1

Notes for Deployment

- The database structure changed (new Event fields and the Remarks history). Run the database update (`migrate`) when deploying.
- Load the default landing-page data once on the server (`backup/auth_routes.json`).
- After deploying, check that events are set up in the Admin page with the correct Campus and dates. Otherwise the Event / Service dropdown will be empty for that campus.

Part 2: Code Changes

These are the changes made to each file this week.

`+ line`	Added
`- line`	Removed
`line`	Unchanged, shown for context
`Lines 10–18`	Where in the file the change is (line numbers in the new version)

Changes that only add or remove spaces or blank lines are left out. Characters that can't be displayed, such as emoji, are shown as "?".

backup/auth_routes.json NEW FILE

Default landing page for each role · +42 / −0 lines

```
Lines 1–42
+ [
+   {
+     "model": "Account.authroute",
+     "pk": 1,
+     "fields": {
+       "name": "Pastor",
+       "code": "PTR",
+       "url": "/approvals_list/",
+       "updated_on": "2026-10-20T00:00:00"
+     }
+   },
+   {
+     "model": "Account.authroute",
+     "pk": 2,
+     "fields": {
+       "name": "FI",
+       "code": "FI",
+       "url": "/",
+       "updated_on": "2026-10-20T00:00:00"
+     }
+   },
+   {
+     "model": "Account.authroute",
+     "pk": 3,
+     "fields": {
+       "name": "Leader",
+       "code": "LE",
+       "url": "/trip",
+       "updated_on": "2026-10-20T00:00:00"
+     }
+   },
+   {
+     "model": "Account.authroute",
+     "pk": 4,
+     "fields": {
+       "name": "Accounting",
+       "code": "ACNT",
+       "url": "/accounting/",
+       "updated_on": "2026-10-20T00:00:00"
+     }
+   }
+ ]
```

cashier/admin.py

Event list in Admin shows the campus · +7 / −5 lines

```
Lines 5–26

  @admin.register(Event)
  class EventAdmin(admin.ModelAdmin):
-     list_display = ['name', 'start_date', 'end_date', 'description']
-     search_fields = ['name', 'start_date', 'end_date', 'description']
+     list_display = ['name', 'campus__name', 'start_date', 'end_date', 'description']
+     search_fields = ['name', 'campus__name', 'start_date', 'end_date', 'description']

  @admin.register(RemarksList)
  class RemarksListAdmin(admin.ModelAdmin):
      list_display = ['description', 'user', 'cashier', 'timestamp']
      search_fields = ['description', 'user__first_name', 'user__last_name', 'cashier__leader__first_name',
   'cashier__leader__last_name']

- @admin.register(Vehicle)
```

```
+ # @admin.register(CampusEvent)
+ # class CampusEventAdmin(admin.ModelAdmin):
+ #     list_display = ['campus__name', 'event__name']
+ #     search_fields = ['campus__name', 'event__name']
```

```
+ @admin.register(Vehicle)
  class VehicleAdmin(admin.ModelAdmin):
-
      list_display = ['name', 'peso_price']
-
      search_fields = ['name']
```

cashier/forms.py

Trip form uses the campus-based event list; “Special Event” removed from Campus choices · +22 / −12 lines

```
Lines 1–4
-
+ from django.utils import timezone

  from django import forms
```

```
Lines 16–26

  from django.core.exceptions import ValidationError

- from django.db.models import Q
+ from django.db.models import Q, F

  from .models import Cashier, Attendees, Event, SubsidyType,Campus, Vehicle, ServiceSchedule, City

-
+ from .utils import events_for_campus
```

```
Lines 40–49

  class CashierEntryForm(forms.ModelForm):
      events = forms.ModelChoiceField(
-         queryset=Event.objects.all(),
+         queryset=Event.objects.none(),
          required=True,
+         label='Event / Service',
+         empty_label='Select Event or Service',
          widget=forms.Select(attrs={'class': 'form-control', 'id': 'id_events'})
      )
```

```
Lines 60–66
          queryset=Campus.objects.all(),
          widget=forms.Select(attrs={
              'class': 'form-control',
-             'id': 'id_campus',
-             'data-action': 'change->trip-form#filterSchedules',
+             'id': 'id_campus'
          })
      )
```

```
Lines 128–155
          }

      def __init__(self, *args, **kwargs):
-
          super().__init__(*args, **kwargs)

-         # Filters schedules based on campus during form validation
-
-         if 'campus' in self.data:
+         # FIX: Remove Service Schedule among the list of Campus
+         self.fields['campus'].queryset = Campus.objects.exclude(name='Special Event')

+         campus_id = None
+         if self.data.get('campus'):
              try:
                  campus_id = int(self.data.get('campus'))
+             except (ValueError, TypeError):
+                 campus_id = None
+
+         if campus_id:
              self.fields['service_schedule'].queryset = ServiceSchedule.objects.filter(campus_id=campus_id)

-             except (ValueError, TypeError):
-                 self.fields['service_schedule'].queryset = ServiceSchedule.objects.none()
```

```
+             event_queryset = events_for_campus(campus_id)
+             self.fields['events'].queryset = event_queryset
+
+             current_event = event_queryset.first()
+             if current_event:
+                 self.fields['events'].initial = current_event.pk
+
```

cashier/models.py

Event: added campus and priority; dates and description are now optional · +18 / −7 lines

```
Lines 8–14
  from django.utils import timezone
  import uuid

- class Event(models.Model):
-     name = models.CharField(max_length=200)
-     description = models.CharField(max_length=200)
-     start_date = models.DateField()
-     end_date = models.DateField()

-     def __str__(self):
-         return self.name


  class Vehicle(models.Model):
Lines 63–89


+ class Event(models.Model):
+     name = models.CharField(max_length=200)
+     description = models.TextField(max_length=200, null=True, blank=True) # FIX 10/19/2026: Updated the field type for
   description from Char to Texts
+     start_date = models.DateField(null=True, blank=True)
+     end_date = models.DateField(null=True, blank=True)
+     campus = models.ForeignKey(Campus, null=True, blank=True, on_delete=models.SET_NULL, related_name='campus')
+     priority = models.PositiveIntegerField(
+         default=0,
+         help_text="Higher number = higher priority"
+     ) # This field will designate the level of priority in the selection of Events, if any same day events occur

+     def __str__(self):
+         return f'{self.name}'

+ # class CampusEvent(models.Model):
+ #   campus =  models.ForeignKey(Campus, on_delete=models.SET_NULL, null=True, blank=True, related_name='campus')
+ #   event = models.ForeignKey(Campus, on_delete=models.SET_NULL, null=True, blank=True, related_name='event')
+ #   uuid = models.UUIDField(unique=True, primary_key=True, default=uuid.uuid4, editable=False)

+ #   def __str__(self):
+ #           return f'{self.campus__name} - {self.event__name}'

  class Attendees(models.Model):
```

cashier/old_forms.py

Removed an unused setting on the Campus field · +0 / −2 lines

```
Lines 68–73

              'id': 'id_campus',

-             'data-action': 'change->trip-form#filterSchedules',
-
          })

      )
```

cashier/remarks_urls.py NEW FILE

Address for submitting a remark · +7 / −0 lines

```
Lines 1–7
+ from django.urls import path
+
+ from . import remarks_views
+
+ urlpatterns = [
+     path('remarks/<int:pk>/submit/', remarks_views.submit_remark, name='submit-remark'),
+ ]
```

cashier/remarks_views.py **NEW FILE**

Saves and lists remarks · +69 / −0 lines

Lines 1–69

```
+ import json
+
+ from django.contrib.auth.decorators import login_required
+ from django.http import JsonResponse
+ from django.shortcuts import get_object_or_404
+ from django.utils import timezone
+ from django.views.decorators.http import require_POST
+
+ from .models import Cashier, RemarksList
+
+ # UserType.type codes allowed to leave a remark: Pastor, Finance (cashier), Accountant
+ REMARK_ROLES = ('PTR', 'FI', 'ACNT')
+
+
+ def serialize_remark(remark):
+     user = remark.user
+     return {
+         'description': remark.description,
+         'timestamp': timezone.localtime(remark.timestamp).isoformat(),
+         'user_name': f'{user.first_name or ""} {user.last_name or ""}'.strip() or user.email,
+     }
+
+
+ def get_role_remarks(cashier):
+     # """Latest remark per role for a Cashier entry: {'PTR': {...} | None, 'FI': ..., 'ACNT': ...}."""
+     result = {role: None for role in REMARK_ROLES}
+     entries = (RemarksList.objects
+                .filter(cashier=cashier, user__isnull=False)
+                .select_related('user')
+                .order_by('timestamp'))
+     for remark in entries:  # ascending, so the latest remark of each role wins
+         if remark.user.type in result:
+             result[remark.user.type] = serialize_remark(remark)
+     return result
+
+
+ @login_required
+ @require_POST
+ def submit_remark(request, pk):
+     """Save the logged-in user's remark for Cashier entry `pk`."""
+     if request.user.type not in REMARK_ROLES:
+         return JsonResponse({'status': 'error', 'message': 'You are not allowed to add remarks'}, status=403)
+
+     try:
+         data = json.loads(request.body)
+     except json.JSONDecodeError:
+         return JsonResponse({'status': 'error', 'message': 'Invalid JSON'}, status=400)
+
+     description = (data.get('remarks') or '').strip()
+     if not description:
+         return JsonResponse({'status': 'error', 'message': 'Remark cannot be empty'}, status=400)
+     if len(description) > 300:
+         return JsonResponse({'status': 'error', 'message': 'Remark must be 300 characters or fewer'}, status=400)
+
+     cashier = get_object_or_404(Cashier, pk=pk)
+
+     # One remark per user per entry. Resubmitting edits it; the timestamp is
+     # refreshed manually because the field is auto_now_add.
+     remark, _ = RemarksList.objects.update_or_create(
+         cashier=cashier, user=request.user,
+         defaults={'description': description, 'timestamp': timezone.now()},
+     )
+
+     return JsonResponse({
+         'status': 'success',
+         'message': 'Remark saved',
+         'role': request.user.type,
+         'remark': serialize_remark(remark),
+     })
```

cashier/urls.py

New addresses for event lookup and remarks · +7 / −1 lines

```
Lines 1–4
- from django.urls import path
+ from django.urls import path, include

  from django.contrib.auth.views import LogoutView

Lines 29–36

      path('update_remarks/<int:pk>/', views.update_remarks, name='update-remarks'),

+     path('', include('cashier.remarks_urls')),
+
      path('update_amount/<int:pk>/', views.update_amount, name='update-amount'),

      path('attendees/', AttendeeView.as_view(), name='manage-attendees'),
Lines 51–60
      path('api/list/', views.PastorApprovals.as_view(), name='cashier_api_list'),
      path('api/list/<int:id>', views.PastorApprovals.as_view(), name='cashier_api_list_specific_campus'),
      path('api/getcampuses/', views.getcampuses, name='get-campuses'),
+
+     # Add event fetching
+     path('api/get-events/', views.load_events, name='get-events'),
+
      path('reports/', ReportsView.as_view(), name='reports-trip'),
      path('reports/<str:week>', ReportsView.as_view(), name='reports-trip'),
```

cashier/utils.py NEW FILE

Picks which events a campus should see · +31 / −0 lines

```
Lines 1–31
+ from django.db.models import Case, IntegerField, Q, Value, When, F
+ from django.utils import timezone
+ from .models import Event
+
+ def events_for_campus(campus_id=None):
+     """Active events for a campus, plus events with no campus (shown for every campus)."""
+     today = timezone.localdate()
+
+     # Case 3: Dated events only show within their range; undated events are always active
+     qs = Event.objects.filter(
+         Q(start_date__lte=today) | Q(start_date__isnull=True),
+         Q(end_date__gte=today)   | Q(end_date__isnull=True),
+     )
+
+     # Case 1 & 2: No campus = every campus; specific campus = that campus only
+     if campus_id:
+         qs = qs.filter(Q(campus_id=campus_id) | Q(campus__isnull=True))
+     else:
+         qs = qs.filter(campus__isnull=True)
+
+     # Case 4: Special events are always shown on top of the selection, regardless of date.
+     qs = qs.annotate(
+         special_rank=Case(
+             When(name__icontains='Special Event', then=Value(0)),
+             default=Value(1),
+             output_field=IntegerField(),
+         )
+     )
+
+     # Special Events (dated) on top, ongoing events (no dates) below. Priority is ignored.
+     return qs.order_by('special_rank', F('start_date').asc(nulls_last=True), 'pk')
```

cashier/views.py

New event lookup for the form; login redirection fixed; Accounting page uses the new dashboard · +23 / −27 lines

```
Lines 35–42
  from openpyxl.worksheet.table import Table, TableStyleInfo
  import openpyxl
  import secrets
+ from .remarks_views import get_role_remarks
+ from .utils import events_for_campus



Lines 667–676

          else:
              # If the user is not authenticated, you might want to redirect them to the login page or show a 404 page. Adjust
   this as needed.
```

```
+             if not user.is_authenticated:
                  redirect_url = reverse('login-cargo')  # Assuming 'login-cargo' is the name of your login URL
-
-             # return render(request, 'error_pages/404_page.html', status=404)
+             else:
+                 return render(request, 'error_pages/404_page.html', status=404)

          return super().dispatch(request, *args, **kwargs)

Lines 1032–1067

          else:
              # If the user is not authenticated, you might want to redirect them to the login page or show a 404 page. Adjust
   this as needed.
+             if not user.is_authenticated:
                  redirect_url = reverse('login-cargo')  # Assuming 'login-cargo' is the name of your login URL

-             # return render(request, 'error_pages/404_page.html', status=404)
+             else:
+                 return render(request, 'error_pages/404_page.html', status=404)

          return super().dispatch(request, *args, **kwargs)



      def get(self, request, id=None, *args, **kwargs):
-
          if id:
-
              return self.get_cashier_detail(request, id)

-
-
          if request.headers.get('X-Requested-With') == 'XMLHttpRequest':
-
              return self.get_cashier_list(request)

-
-
          form = ReportDateRangeFilterForm()
-
-         return render(request, 'Accountant/index.html', {'form': form})
-
-
-
-
+         return render(request, 'Accountant/index-2.html', {'form': form})

      def post(self, request, id=None, *args, **kwargs):
-
          form = ReportDateRangeFilterForm(request.POST)

-
-
+         print(f'Response: {form}')
          if form.is_valid():
-
-
-
              start_date = form.cleaned_data.get('start_date')
-
              end_date = form.cleaned_data.get('end_date')
-
              user = self.request.user

-
-
              start_date_format = start_date.strftime('%B %d, %Y')
-
              end_date_format = end_date.strftime('%B %d, %Y')

              response = HttpResponse(content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet')
Lines 1300–1307

                  'remarks': cashier.remarks,

+                 'role_remarks': get_role_remarks(cashier),
+
                  'time_arrived': cashier.time_arrived.isoformat() if cashier.time_arrived else None,

                  'images': [{'url': request.build_absolute_uri(img.image.url), 'uploaded_at': img.uploaded_at.isoformat()} for

   img in cashier.images.all()],
Lines 2169–2182
          return JsonResponse({'status':'success', 'campus':user.campus.name,'id': user.campus.id})
```

```
      except Exception as e:
          return JsonResponse({'status':'error', 'message':str(e)})
+
+ # 20261020: To trigger the fetching of Events based on chosen campus
+ @login_required
+ def load_events(request):
+     try:
+         campus_id = int(request.GET.get('campus_id'))
+     except (TypeError, ValueError):
+         campus_id = None
+
+     events = [{'id': e.pk, 'name': str(e)} for e in events_for_campus(campus_id)]
+     return JsonResponse({'events': events})
```

templates/Accountant/base.html NEW FILE

Page layout for Accountant pages · +33 / −0 lines

```
Lines 1–33
+ <!doctype html>
+ <html>
+ <!--
+     WARNING! Make sure that you match all Quasar related
+     tags to the same version! (Below it's "@2.34.0")
+   -->
+
+ <head>
+     <title>{% block title %} {% endblock%}</title>
+     <link
+         href="https://fonts.googleapis.com/css?family=Roboto:100,300,400,500,700,900|Material+Icons|Material+Icons+Outlined|M
a
   terial+Icons+Round|Material+Icons+Sharp"
+         rel="stylesheet" type="text/css">
+     <link href="https://cdn.jsdelivr.net/npm/ionicons@^4.0.0/dist/css/ionicons.min.css" rel="stylesheet"
+         type="text/css">
+     <link href="https://cdn.jsdelivr.net/npm/eva-icons@^1.0.0/style/eva-icons.css" rel="stylesheet" type="text/css">
+     <link href="https://maxst.icons8.com/vue-static/landings/line-awesome/line-awesome/1.3.0/css/line-awesome.min.css"
+         rel="stylesheet" type="text/css">
+     <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@^1.0.0/font/bootstrap-icons.css" rel="stylesheet"
+         type="text/css">
+     <link href="https://cdn.jsdelivr.net/npm/animate.css@^4.0.0/animate.min.css" rel="stylesheet" type="text/css">
+     <link href="https://cdn.jsdelivr.net/npm/quasar@2.34.0/dist/quasar.prod.css" rel="stylesheet" type="text/css">
+
+     {% include 'Accountant/styles.html' %}
+ </head>
+
+ <body>
+
+             {% block content %}
+
+             {% endblock %}
+
+ </body>
+ </html>
```

templates/Accountant/entry-detail.html NEW FILE

Full details of a trip entry · +107 / −0 lines

```
Lines 1–107
+ <div class="modal-body-scroll">
+     <div class="detail-section"
+          :style="{ background: statusBg, borderLeft: '5px solid ' + statusColor }">
+         <div class="d-flex justify-content-between align-items-center mb-3">
+             <h5 :style="{ color: statusColor, margin: 0 }">Payment Summary</h5>
+             <q-badge :color="statusColor">[[ statusText ]]</q-badge>
+         </div>
+         <div class="detail-grid">
+             <div class="detail-item">
+                 <span class="detail-label">Subsidy Amount</span>
+                 <span class="detail-value" :style="bigValueStyle">
+                     ₱ [[ Number(selectedRow.subsidy_amount ?? 0).toLocaleString(undefined, { minimumFractionDigits: 2 }) ]]
+                 </span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Amount Released</span>
+                 <span class="detail-value" :style="bigValueStyle">
+                     ₱ [[ Number(selectedRow.amount_released ?? 0).toLocaleString(undefined, { minimumFractionDigits: 2 }) ]]
+                 </span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Voucher Code</span>
+                 <span class="detail-value" :style="bigValueStyle">[[ selectedRow.voucher ]]</span>
+             </div>
+         </div>
+     </div>
```

```
+
+     <div class="detail-section">
+         <h5>Trip Information</h5>
+         <div class="detail-grid">
+             <div class="detail-item">
+                 <span class="detail-label">Leader</span>
+                 <span class="detail-value">
+                     [[ selectedRow.leader ? selectedRow.leader.first_name + ' ' + selectedRow.leader.last_name : 'N/A' ]]
+                 </span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Date</span>
+                 <span class="detail-value">[[ formatDate(selectedRow.date) ]]</span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Origin</span>
+                 <span class="detail-value">[[ selectedRow.start_location?.name ]]</span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Destination</span>
+                 <span class="detail-value">[[ selectedRow.campus?.name ]]</span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Subsidy Type</span>
+                 <span class="detail-value">[[ selectedRow.subsidy_type?.name ]]</span>
+             </div>
+             <div class="detail-item" style="grid-column: span 2;">
+                 <span class="detail-label">Service Schedule</span>
+                 <span class="detail-value" style="color: #667eea; font-weight: 700;">[[ formattedSchedule ]]</span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Vehicle</span>
+                 <span class="detail-value">[[ selectedRow.vehicle ? selectedRow.vehicle.name : 'N/A' ]]</span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Plate Number</span>
+                 <span class="detail-value">[[ selectedRow.plate_num || 'N/A' ]]</span>
+             </div>
+             <div class="detail-item">
+                 <span class="detail-label">Remarks</span>
+                 <span class="detail-value">[[ selectedRow.remarks || 'N/A' ]]</span>
+             </div>
+             <div class="detail-item" v-if="selectedRow.time_arrived">
+                 <span class="detail-label">Actual Arrival</span>
+                 <span class="detail-value">[[ formatDateTime(selectedRow.time_arrived) ]]</span>
+             </div>
+         </div>
+     </div>
+
+     <div class="detail-section">
+         <h5>Passengers ([[ selectedRow.users ? selectedRow.users.length : 0 ]])</h5>
+         <div class="passengers-list">
+             <div class="passenger-grid" v-if="selectedRow.users && selectedRow.users.length > 0">
+                 <div class="passenger-card" v-for="u in selectedRow.users" :key="u.cop_id">
+                     <div class="passenger-info">
+                         <div class="passenger-name">
+                             COPID: [[ u.cop_id ]] - Name: [[ u.first_name ]] [[ u.middle_name ]] [[ u.last_name ]]
+                         </div>
+                     </div>
+                 </div>
+             </div>
+             <div class="no-passengers" v-else>No passengers recorded</div>
+         </div>
+     </div>
+
+     <div class="detail-section text-center">
+         <h5>Photos ([[ hasImages ? selectedRow.images.length : 0 ]])</h5>
+         <div v-if="hasImages">
+             <div class="photo-item" v-for="(img, idx) in selectedRow.images" :key="idx">
+                 <q-img :src="img.url" class="img-fluid rounded voucher-preview"
+                      @click="openImage(img.url)">
+                     <div class="absolute-bottom text-subtitle1 text-center">
+           [[ formatDateTime(img.uploaded_at) ]]
+         </div>
+                 </q-img>
+
+             </div>
+         </div>
+         <div class="no-photo-placeholder" v-else>No photos uploaded</div>
+     </div>
+ </div>
```

templates/Accountant/index-2.html NEW FILE

Redesigned Accountant dashboard · +411 / −0 lines

Lines 1–411

```
+ {% extends 'Accountant/base.html '%}
+ {% block title %}Accountant Dashboard{% endblock %}
+
+ {% load static %}
+ {% block content %}
+     <!-- example of injection point where you write your app template -->
+     <div id="q-app">
+         <q-layout view="hHh lpR fFf">
+             <q-header reveal bordered class="bg-white text-green q-pa-md" height-hint="98">
+                 <q-toolbar>
+                     <q-img bordered width="96px" height="auto" alt="Quasar logo"
+                         src="{% static 'img/cargo_logo.png'%}"></q-img> |
+                     <q-toolbar-title>
+                         Accounting
+                     </q-toolbar-title>
+
+                     <form action="{% url 'logout-cargo' %}" method="post" class="q-ml-sm" style="margin: 0;">
+                         {% csrf_token %}
+                         <q-btn color="negative" no-caps icon="logout" label="Logout" type="submit"></q-btn>
+                     </form>
+                 </q-toolbar>
+
+                 <q-btn push color="green" class="q-ma-md" label="Refresh Data" @click="fetchCashierData"></q-btn>
+
+
+             </q-header>
+             <q-page-container>
+                 <q-page padding>
+
+                     <q-card bordered class="my-card q-mb-md">
+                         <q-card-section>
+                             <div class="text-h6 ">Vouchers: <span v-text="count"></span></div>
+                         </q-card-section>
+
+                         <q-card-section>
+                             <form method="POST" action="{% url 'accounting' %}"
+                                 class="row items-end q-col-gutter-md q-mb-md">
+                                 {% csrf_token %}
+
+                                 <div class="col-12 col-sm-auto">
+                                     <q-input v-model="startDate" name="start_date" type="date" label="Start date"
+                                         stack-label outlined dense></q-input>
+                                 </div>
+
+                                 <div class="col-12 col-sm-auto">
+                                     <q-input v-model="endDate" name="end_date" type="date" label="End date" stack-label
+                                         outlined dense :min="startDate"></q-input>
+                                 </div>
+
+                                 <div class="col-12 col-sm-auto">
+                                     <q-btn type="submit" color="positive" icon="download" label="Export" unelevated
+                                         no-caps></q-btn>
+                                 </div>
+                             </form>
+
+                         </q-card-section>
+
+
+                         <q-separator dark inset />
+                     </q-card>
+
+
+
+                     <q-table :rows="rows" :columns="columns" row-key="row_num" :loading="loading" flat bordered
+                         v-model:pagination="pagination" :filter="filter" :rows-per-page-options="[10, 25, 50, 100]"
+                         rows-per-page-label="Show entries:" :pagination-label="paginationLabel"
+                         no-data-label="No data available in table" no-results-label="No matching records found">
+                         <template v-slot:body-cell-completed="props">
+                             <!-- For transforming the Completed Status to an icon-->
+                             <q-td :props="props">
+                                 <q-icon :name="props.value ? 'check_circle' : 'cancel'"
+                                     :color="props.value ? 'positive' : 'negative'" size="sm"></q-icon>
+                             </q-td>
+
+                         </template>
+
+                         <!-- Search bar -->
+                         <template #top-right>
+                             <q-input outlined dense debounce="300" v-model="filter" placeholder="Search">
```

```
+                                <template #append>
+                                    <q-icon name="search" />
+                                </template>
+                            </q-input>
+                        </template>
+
+                        <!-- Actions column: buttons -->
+                        <template v-slot:body-cell-actions="props">
+                            <q-td :props="props">
+                                <div class="q-pa-md row q-gutter-md items-start">
+
+                                    <q-btn unelevated color="primary" icon="visibility" label="View"
+                                        @click="viewDetail(props.row)"></q-btn>
+
+                                    <q-btn outline color="info" icon="comment" label="Remarks"
+                                        @click="openRemarksModal(props.row)"></q-btn>
+
+                                </div>
+
+                                <!-- For modals-->
+
+                                <!-- View detail modal -->
+                                <q-dialog v-model="showDetail">
+                                    <q-card style="min-width: 50%">
+                                        <q-card-section class="bg-green row items-center q-pb-md">
+                                            <div class="text-h6">Entry Details</div>
+                                            <q-space></q-space>
+                                            <q-btn icon="close" flat round dense v-close-popup></q-btn>
+                                        </q-card-section>
+
+                                        <q-card-section>
+                                            <!-- show whatever fields your row has -->
+
+                                            {% include 'Accountant/entry-detail.html' %}
+
+                                        </q-card-section>
+                                    </q-card>
+                                </q-dialog>
+
+                                <!-- Remarks modal -->
+                                <q-dialog v-model="showRemarks" persistent transition-show="scale"
+                                    transition-hide="scale">
+                                    {% include 'Accountant/remarks.html' %}
+                                </q-dialog>
+
+    </div>
+    </q-td>
+
+
+
+
+    </template>
+    </q-table>
+    </q-page>
+    </q-page-container>
+
+    </q-layout>
+    </div>
+
+    <!-- Add the following at the end of your body tag -->
+    <script src="https://cdn.jsdelivr.net/npm/vue@3/dist/vue.global.prod.js"></script>
+    <script src="https://cdn.jsdelivr.net/npm/quasar@2.34.0/dist/quasar.umd.prod.js"></script>
+    <script src="https://unpkg.com/vue-router@4"></script>
+
+    <script>
+        const { useQuasar, date } = Quasar
+        const { ref, onMounted, computed } = Vue
+        const filter = ref('')
+
+        const app = Vue.createApp({
+            setup() {
+                const $q = useQuasar()
+
+                // For modals
+                const showDetail = ref(false)
+                const showRemarks = ref(false)
+                const selectedRow = ref({})
+                const remarks = ref('')
+                const remarksId = ref(null)
+                const loadingRemarks = ref(false)
+                const savingRemarks = ref(false)
+
+                // Remark roles: only the logged-in user's role can be edited
+                const currentUserRole = '{{ request.user.type }}'
+                const remarkRoles = [
```

```
+                    { code: 'PTR', title: 'Pastor' },
+                    { code: 'FI', title: 'Cashier' },
+                    { code: 'ACNT', title: 'Accountant' },
+                ]
+                const canRemark = remarkRoles.some(r => r.code === currentUserRole)
+                const roleRemarks = ref({})
+                const roleDrafts = ref({ PTR: '', FI: '', ACNT: '' })
+
+
+                const startDate = ref('{{ form.start_date.value|default_if_none:"" }}')
+                const endDate = ref('{{ form.end_date.value|default_if_none:"" }}')
+
+
+                //For the tables
+                const rows = ref([])
+                const count = ref(0)
+                const loading = ref(false)
+                const pagination = ref({ sortBy: null, descending: false, page: 1, rowsPerPage: 10 })
+
+                // For the formatting of the money
+                const peso = val => `₱${Number(val ?? 0).toLocaleString(undefined, { minimumFractionDigits: 2 })}`
+                const dateFormat = (val) => val ? date.formatDate(val, 'MMMM D, YYYY hh:mm:ss A') : '';
+
+
+                const columns = [
+                    { name: 'row_num', label: 'Row', field: 'row_num', sortable: true, align: 'left' },
+                    { name: 'leader.last_name', label: 'Leader Name', field: row => `${row.leader?.first_name ?? ''}
   ${row.leader?.last_name ?? ''}`.trim(), sortable: true, align: 'left' },
+                    {
+                        name: 'date_time_arrived', label: 'Date and Time Arrived', field: 'date_time_arrived', align: 'right'
,
    sortable: true,
+                        format: dateFormat, sort: (a, b) => new Date(a).getTime() - new Date(b).getTime()
+                    },
+                    { name: 'campus', label: 'Campus', field: row => row.campus?.name ?? '', align: 'right', sortable: true }
,
+                    { name: 'subsidy_amount', label: 'Subsidy Amount (PhP)', field: 'subsidy_amount', align: 'right',
  sortable: true, format: peso },
+                    { name: 'amount_released', label: 'Amount Released (PhP)', field: 'amount_released', align: 'right',
  sortable: true, format: peso },
+                    { name: 'completed', label: 'Completed', field: 'completed', align: 'right', sortable: true },
+                    { name: 'voucher', label: 'Voucher', field: 'voucher', align: 'right', sortable: true },
+                    { name: 'actions', label: 'Actions', field: 'id', align: 'center' }
+                ]
+
+
+
+                async function fetchCashierData() {
+                    loading.value = true
+                    try {
+                        const response = await fetch(`/fetch_vouchers/`, {
+                            headers: { 'X-Requested-With': 'XMLHttpRequest' }
+                        })
+
+                        if (response.status === 401 || response.status === 403) {
+                            showError('Session expired. Please login again.')
+                            window.location.href = '/login/'
+                            return
+                        }
+
+                        const contentType = response.headers.get('content-type')
+                        if (!contentType || !contentType.includes('application/json')) {
+                            console.error('Expected JSON but got:', contentType)
+                            showError('Server returned an unexpected response. Please refresh manually.')
+                            return
+                        }
+
+                        const data = await response.json()
+                        if (data.status === 'success') {
+                            count.value = data.count
+                            rows.value = data.data
+                        }
+                    } catch (error) {
+                        console.error(error)
+                        showError('Failed to load data. Please check your connection.')
+                    } finally {
+                        loading.value = false
+                    }
+                }
+
+
+                async function viewDetail(row) {
+                    const id = row.id
+
+                    const data = await fetch(`/accounting/${id}/details/`, {
```

```
+                        method: 'GET',
+                        headers: {
+                            'Content-Type': 'application/json',
+                            'X-Requested-With': 'XMLHttpRequest'
+                        }
+                    }).then(response => {
+                        if (!response.ok) {
+                            throw new Error('Failed to fetch details')
+                        }
+                        return response.json()
+                    }).then(result => {
+                        if (result.status !== 'success') {
+                            throw new Error(result.message || 'Request failed')
+                        }
+                        console.log('Result', result)
+                        selectedRow.value = result.data
+                        showDetail.value = true
+                    }).catch(err => {
+                        $q.notify({ type: 'negative', message: err.message })
+                    })
+
+                    showDetail.value = true
+                }
+
+                // Fetch the latest remarks for the targeted entry when the dialog opens
+                async function openRemarksModal(row) {
+                    selectedRow.value = row
+                    remarksId.value = row.id
+                    remarks.value = ''
+                    roleRemarks.value = {}
+                    roleDrafts.value = { PTR: '', FI: '', ACNT: '' }
+                    showRemarks.value = true
+                    loadingRemarks.value = true
+                    try {
+                        const response = await fetch(`/accounting/${row.id}/details/`, {
+                            headers: { 'X-Requested-With': 'XMLHttpRequest' }
+                        })
+                        const result = await response.json()
+                        if (!response.ok || result.status !== 'success') {
+                            throw new Error(result.message || 'Failed to fetch remarks')
+                        }
+                        remarks.value = result.data.remarks ?? ''
+                        roleRemarks.value = result.data.role_remarks ?? {}
+                        for (const r of remarkRoles) {
+                            roleDrafts.value[r.code] = roleRemarks.value[r.code]?.description ?? ''
+                        }
+                    } catch (err) {
+                        showRemarks.value = false
+                        showError(err.message)
+                    } finally {
+                        loadingRemarks.value = false
+                    }
+                }
+
+                function getCsrfToken() {
+                    return document.querySelector('[name=csrfmiddlewaretoken]')?.value
+                        || decodeURIComponent((document.cookie.match(/csrftoken=([^;]+)/) || [])[1] || '')
+                }
+
+                async function saveRemarks() {
+                    savingRemarks.value = true
+                    try {
+                        const response = await fetch(`/remarks/${remarksId.value}/submit/`, {
+                            method: 'POST',
+                            headers: { 'Content-Type': 'application/json', 'X-CSRFToken': getCsrfToken() },
+                            body: JSON.stringify({ remarks: roleDrafts.value[currentUserRole] })
+                        })
+                        const result = await response.json()
+                        if (!response.ok || result.status !== 'success') throw new Error(result.message)
+                        // Reflect the saved remark, its time and its author right in the dialog
+                        roleRemarks.value = { ...roleRemarks.value, [result.role]: result.remark }
+                        roleDrafts.value[result.role] = result.remark.description
+                        $q.notify({ type: 'positive', message: 'Remark saved' })
+                    } catch (err) {
+                        showError('Failed to save remarks: ' + err.message)
+                    } finally {
+                        savingRemarks.value = false
+                    }
+                }
+
+                function parseDate(r) {
+                    return new Date(r.created_at).toISOString()
+                }
+
+                function showError(message) {
```

```
+                    $q.notify({ type: 'negative', message, position: 'top' })
+                }
+
+                onMounted(fetchCashierData)
+
+                // For the dialog / modal
+
+                const hasImages = computed(() => !!selectedRow.value?.images?.length)
+
+                // Adjust this to whatever your original status logic was
+                const amountReleased = computed(() => Number(selectedRow.value?.amount_released ?? 0))
+
+                const status = computed(() => {
+                    const d = selectedRow.value || {}
+                    if (d.rejected) {
+                        return { color: '#e53e3e', bg: '#fff5f5', text: 'REJECTED' }
+                    } else if (d.is_late) {
+                        return { color: '#718096', bg: '#f7fafc', text: 'LATE' }
+                    } else if (amountReleased.value > 0) {
+                        return { color: '#38a169', bg: '#f0fff4', text: 'APPROVED' }
+                    }
+                    return { color: '#d69e2e', bg: '#fffaf0', text: 'PENDING' }
+                })
+
+                const formattedSchedule = computed(() => {
+                    const d = selectedRow.value || {}
+                    const schedule = d.service_schedule
+                    if (!schedule) return ''
+
+                    const campusName = d.campus ? d.campus.name : 'N/A'
+                    const scheduleDay = schedule.day || ''
+
+                    let timeStr = schedule.time
+                    if (timeStr) {
+                        const [hours, minutes] = timeStr.split(':')
+                        let hour = parseInt(hours, 10)
+                        const ampm = hour >= 12 ? 'PM' : 'AM'
+                        hour = hour % 12 || 12
+                        timeStr = `${hour}:${minutes} ${ampm}`
+                    }
+
+                    return `${campusName} | ${scheduleDay} ${timeStr ?? ''}`.trim()
+                })
+
+                const statusColor = computed(() => status.value.color)
+                const statusBg = computed(() => status.value.bg)
+                const statusText = computed(() => status.value.text)
+
+                const bigValueStyle = computed(() => ({
+                    fontSize: '1.6rem', fontWeight: 800, color: statusColor.value
+                }))
+
+                const formatDate = val => val ? date.formatDate(val, 'MMMM D, YYYY') : 'N/A'
+                const formatDateTime = val => val ? date.formatDate(val, 'MMMM D, YYYY hh:mm:ss A') : 'N/A'
+
+                const openImage = url => window.open(url, '_blank')
+
+                return {
+                    rows, columns, count, loading, pagination, fetchCashierData, startDate, endDate, filter,
+                    showDetail,
+                    showRemarks,
+                    selectedRow,
+                    remarks, loadingRemarks, savingRemarks,
+                    currentUserRole, remarkRoles, canRemark, roleRemarks, roleDrafts,
+                    viewDetail,
+                    openRemarksModal,
+                    saveRemarks,
+                    formatDate, formatDateTime,
+                    hasImages, statusColor, statusBg, statusText, bigValueStyle, formattedSchedule, openImage
+                }
+            }
+        })

+        app.use(Quasar, {
+            config: {},
+            plugins: { Notify: Quasar.Notify, }
+        })

+        app.config.compilerOptions.delimiters = ['[[', ']]']
+        app.mount('#q-app')
+    </script>

+ {% endblock %}
```

templates/Accountant/index.html

Fixed the Export link · +1 / −1 lines

```
Lines 983–989
          <div data-cashier-target="error" class="error hidden"></div>

          <div class="table-wrapper">
-             <form class="d-flex align-items-end gap-3 mb-3" method="POST" action="{% url 'cashier' %}">
+             <form class="d-flex align-items-end gap-3 mb-3" method="POST" action="{% url 'accounting' %}">
                  {% csrf_token %}
                  <div>
                      {{ form.start_date.label_tag }}
```

templates/Accountant/remarks.html NEW FILE

Remarks history section · +30 / −0 lines

```
Lines 1–30
+ <q-card style="min-width: 50%">
+   <q-card-section>
+     <div class="text-h6">Remarks</div>
+   </q-card-section>
+
+   <q-card-section>
+     <h6 class="detail-label">System Generated Remarks</h6>
+     <q-input v-model="remarks" type="textarea" outlined autogrow
+              label="System Generated Remarks" :disable="loadingRemarks" readonly></q-input>
+   </q-card-section>
+
+   <!-- One section per role. Only the section matching the logged-in user's role is editable. -->
+   <q-card-section v-for="r in remarkRoles" :key="r.code">
+     <h6 class="detail-value">[[ r.title ]]'s Remark</h6>
+     <small class="detail-label">
+       Last updated:
+       [[ roleRemarks[r.code] ? formatDateTime(roleRemarks[r.code].timestamp) + ' by ' + roleRemarks[r.code].user_name : 'N/A'

   ]]
+     </small>
+     <q-input v-model="roleDrafts[r.code]" type="textarea" outlined autogrow maxlength="300"
+              :label="r.title + '\'s Remark'"
+              :readonly="r.code !== currentUserRole"
+              :disable="loadingRemarks || savingRemarks"
+              :bg-color="r.code === currentUserRole ? 'white' : 'grey-2'"></q-input>
+   </q-card-section>
+
+   <q-card-actions align="right">
+     <q-btn flat label="Close" v-close-popup :disable="savingRemarks"></q-btn>
+     <q-btn v-if="canRemark" color="positive" label="Submit" :loading="savingRemarks" @click="saveRemarks"></q-btn>
+   </q-card-actions>
+ </q-card>
```

templates/Accountant/styles.html NEW FILE

Styling for Accountant pages · +352 / −0 lines

```
Lines 1–352
+ <style>
+         * {
+             margin: 0;
+             padding: 0;
+             box-sizing: border-box;
+         }
+
+         body {
+             font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, sans-serif;
+             background: linear-gradient(135deg, #38a169 0%, #276749 100%);
+             min-height: 100vh;
+             padding: 20px;
+         }
+
+         .container {
+             max-width: 1600px;
+             margin: 0 auto;
+         }
+
+         .header {
+             background: white;
+             padding: 30px;
+             border-radius: 12px;
+             box-shadow: 0 4px 6px rgba(0,0,0,0.1);
+             margin-bottom: 30px;
+         }
+
+         .header-top {
+             display: flex;
```

```
+             justify-content: space-between;
+             align-items: center;
+             margin-bottom: 15px;
+         }
+
+         .header-actions {
+             display: flex;
+             gap: 10px;
+             align-items: center;
+             width: 100%;
+         }
+
+         .header-actions form {
+             margin-left: auto !important;
+         }
+
+         h1 {
+             color: #333;
+             font-size: 28px;
+         }
+
+         .count-badge {
+             background: #38a169;
+             color: white;
+             padding: 8px 16px;
+             border-radius: 20px;
+             font-weight: 600;
+             font-size: 14px;
+         }
+
+         .refresh-btn, .logout-btn {
+             background: #38a169;
+             color: white;
+             border: none;
+             padding: 10px 20px;
+             border-radius: 6px;
+             cursor: pointer;
+             font-size: 14px;
+             font-weight: 600;
+             transition: background 0.3s;
+         }
+
+         .logout-btn {
+             background: #e53e3e;
+         }
+
+         .refresh-btn:hover { background: #5568d3; }
+         .logout-btn:hover { background: #c53030; }
+
+         .loading, .error {
+             background: white;
+             padding: 20px;
+             border-radius: 8px;
+             text-align: center;
+             margin-bottom: 20px;
+         }
+
+         .loading { color: #667eea; font-weight: 600; }
+         .error { color: #e53e3e; background: #fff5f5; border: 1px solid #fc8181; }
+
+         .hidden { display: none; }
+
+         .table-wrapper {
+             background: white;
+             padding: 30px;
+             border-radius: 12px;
+             box-shadow: 0 4px 6px rgba(0,0,0,0.1);
+         }
+
+         table.dataTable { width: 100% !important; }
+
+         /* ── Completed badges ── */
+         .completed-badge {
+             display: inline-flex;
+             align-items: center;
+             justify-content: center;
+             width: 28px;
+             height: 28px;
+             border-radius: 50%;
+             font-size: 15px;
+             font-weight: 800;
+             cursor: default;
+         }
+         .completed-yes {
+             background: #c6f6d5;
```

```
+            color: #276749;
+            border: 2px solid #68d391;
+        }
+        .completed-no {
+            background: #fed7d7;
+            color: #9b2335;
+            border: 2px solid #fc8181;
+        }
+
+        /* ── Action Buttons ── */
+        .view-detail-btn {
+            background: #48bb78;
+            color: white;
+            border: none;
+            padding: 6px 12px;
+            border-radius: 6px;
+            cursor: pointer;
+            font-size: 12px;
+            font-weight: 600;
+            transition: all 0.3s;
+        }
+        .view-detail-btn:hover {
+            background: #38a169;
+            transform: translateY(-1px);
+        }
+
+        .btn-voucher-verify {
+            background: #553c9a;
+            color: white;
+            border: none;
+            padding: 4px 10px;
+            border-radius: 6px;
+            cursor: pointer;
+            font-size: 12px;
+            font-weight: 600;
+            transition: all 0.3s;
+        }
+        .btn-voucher-verify:hover:not(:disabled) {
+            background: #44337a;
+            transform: translateY(-1px);
+            box-shadow: 0 2px 6px rgba(85,60,154,0.35);
+        }
+        .btn-voucher-verify.btn-completed {
+            background: #276749;
+            cursor: not-allowed;
+            opacity: 0.7;
+        }
+
+        /* ── Modals ── */
+        .modal {
+            display: none;
+            position: fixed;
+            top: 0; left: 0;
+            width: 100%; height: 100%;
+            background: rgba(0,0,0,0.75);
+            z-index: 10000;
+            align-items: center;
+            justify-content: center;
+            animation: fadeIn 0.2s;
+        }
+        @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
+        .modal.show { display: flex; }
+
+        .modal-content {
+            background: white;
+            padding: 0;
+            border-radius: 16px;
+            max-width: 800px;
+            width: 90%;
+            max-height: 85vh;
+            overflow: hidden;
+            position: relative;
+            box-shadow: 0 20px 60px rgba(0,0,0,0.3);
+            animation: slideUp 0.3s;
+        }
+        @keyframes slideUp {
+            from { transform: translateY(20px); opacity: 0; }
+            to { transform: translateY(0); opacity: 1; }
+        }
+
+        .modal-content h3 {
+            margin: 0;
+            padding: 25px 30px;
+            font-size: 24px;
+            border-bottom: 2px solid #e2e8f0;
```

```
+            background: linear-gradient(135deg, #38a169 0%, #276749 100%);
+            color: white;
+        }
+
+        .modal-loading, .modal-error { padding: 40px; text-align: center; color: #667eea; font-weight: 600; }
+        .modal-error { color: #e53e3e; }
+
+        .detail-section {
+            padding: 25px 30px;
+            border-bottom: 1px solid #e2e8f0;
+        }
+        .detail-section:last-child { border-bottom: none; }
+        .detail-section h5 { color: #667eea; margin-bottom: 15px; font-size: 18px; font-weight: 700; }
+
+        .detail-grid {
+            display: grid;
+            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
+            gap: 15px;
+        }
+        .detail-item { display: flex; flex-direction: column; gap: 5px; }
+        .detail-label { font-size: 12px; color: #718096; font-weight: 600; text-transform: uppercase; letter-spacing: 0.5px;
}
+        .detail-value { font-size: 15px; color: #2d3748; font-weight: 500; }
+
+        .photo-gallery {
+            display: grid;
+            grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
+            gap: 20px;
+            margin-top: 20px;
+        }
+        .photo-item {
+            background: #f8fafc; padding: 12px; border-radius: 12px;
+            border: 2px solid #e2e8f0; transition: all 0.3s;
+        }
+        .photo-item:hover { border-color: #667eea; transform: translateY(-2px); box-shadow: 0 4px 12px rgba(102,126,234,0.15)
;
    }
+        .voucher-preview { width: 100%; height: 200px; object-fit: cover; cursor: zoom-in; border: 1px solid #cbd5e0; border-
   radius: 8px; }
+        .photo-timestamp { margin: 8px 0 0 0; font-size: 11px; color: #718096; text-align: center; }
+        .no-photo-placeholder { text-align: center; padding: 30px; background: #f7fafc; border-radius: 12px; color: #a0aec0;
   font-style: italic; border: 1px solid #e2e8f0; }
+
+        .close-btn {
+            position: absolute; top: 20px; right: 20px;
+            background: rgba(255,255,255,0.3); color: white;
+            padding: 8px 14px; font-size: 14px;
+            border: 2px solid rgba(255,255,255,0.5);
+            border-radius: 6px; cursor: pointer; font-weight: 700;
+            transition: all 0.2s; z-index: 1;
+        }
+        .close-btn:hover { background: rgba(255,255,255,0.5); transform: scale(1.05); }
+
+        .dataTables_wrapper .dataTables_filter input {
+            border: 2px solid #e2e8f0; border-radius: 6px; padding: 8px 12px; margin-left: 8px;
+        }
+        .dataTables_wrapper .dataTables_length select {
+            border: 2px solid #e2e8f0; border-radius: 6px; padding: 6px 10px; margin: 0 8px;
+        }
+        table.dataTable thead th { background: #f7fafc; color: #2d3748; font-weight: 700; padding: 12px 8px; border-bottom:
   2px solid #e2e8f0; }
+        table.dataTable tbody td { padding: 12px 8px; vertical-align: middle; }
+        table.dataTable tbody tr:hover { background: #f7fafc; }
+
+        .modal-body-scroll {
+            max-height: 70vh; overflow-y: auto; padding-bottom: 20px;
+        }
+        .modal-body-scroll::-webkit-scrollbar { width: 8px; }
+        .modal-body-scroll::-webkit-scrollbar-track { background: #f1f1f1; border-radius: 10px; }
+        .modal-body-scroll::-webkit-scrollbar-thumb { background: #667eea; border-radius: 10px; }
+
+        /* ── Voucher Modal Specific ── */
+        .voucher-modal-body { padding: 30px; }
+
+        .voucher-input-wrap {
+            position: relative;
+            margin: 20px 0 8px;
+        }
+        .voucher-input-wrap input {
+            width: 100%;
+            padding: 14px 16px;
+            font-size: 20px;
+            font-weight: 700;
+            letter-spacing: 4px;
+            text-align: center;
```

```
+             text-transform: uppercase;
+             border: 3px solid #e2e8f0;
+             border-radius: 10px;
+             outline: none;
+             transition: border-color 0.2s, box-shadow 0.2s;
+         }
+         .voucher-input-wrap input:focus {
+             border-color: #667eea;
+             box-shadow: 0 0 0 3px rgba(102,126,234,0.2);
+         }
+         .voucher-hint {
+             font-size: 12px;
+             color: #a0aec0;
+             text-align: center;
+             margin-bottom: 20px;
+         }
+         .voucher-error {
+             color: #e53e3e;
+             background: #fff5f5;
+             border: 1px solid #fc8181;
+             padding: 10px 14px;
+             border-radius: 8px;
+             font-size: 14px;
+             font-weight: 600;
+             margin-bottom: 12px;
+         }
+         .voucher-success {
+             color: #276749;
+             background: #c6f6d5;
+             border: 1px solid #68d391;
+             padding: 10px 14px;
+             border-radius: 8px;
+             font-size: 14px;
+             font-weight: 600;
+             margin-bottom: 12px;
+         }
+         .voucher-submit-btn {
+             width: 100%;
+             background: linear-gradient(135deg, #38a169 0%, #276749 100%);
+             color: white;
+             border: none;
+             padding: 14px;
+             font-size: 16px;
+             font-weight: 700;
+             border-radius: 10px;
+             cursor: pointer;
+             transition: opacity 0.2s, transform 0.15s;
+         }
+         .voucher-submit-btn:hover:not(:disabled) { opacity: 0.9; transform: translateY(-1px); }
+         .voucher-submit-btn:disabled { opacity: 0.6; cursor: not-allowed; }
+         .voucher-cancel-btn {
+             width: 100%;
+             background: #f7fafc;
+             color: #718096;
+             border: 2px solid #e2e8f0;
+             padding: 12px;
+             font-size: 14px;
+             font-weight: 600;
+             border-radius: 10px;
+             cursor: pointer;
+             margin-top: 8px;
+             transition: background 0.2s;
+         }
+         .voucher-cancel-btn:hover { background: #edf2f7; }
+
+         @keyframes shake {
+             0%, 100% { transform: translateX(0); }
+             20% { transform: translateX(-8px); }
+             40% { transform: translateX(8px); }
+             60% { transform: translateX(-5px); }
+             80% { transform: translateX(5px); }
+         }
+         .input-shake { animation: shake 0.4s ease-in-out; border-color: #fc8181 !important; }
+     </style>
```

templates/cargo_form.html

Event dropdown reloads when the campus changes · +14 / −1 lines

```
Lines 46–52
                            </div>
                            <div class="col-md-6">
                                <label class="form-label">{{ form.campus.label }}</label>
-                               <div data-action="change->trip-form#handleCampusChange">
+                               <div data-action="change->trip-form#handleCampusChange change->trip-form#filterEvents">
                                    {{ form.campus }}
                                </div>
                            </div>
Lines 283–299
                this.updateSubmitButton();
            }

+           async filterEvents(event) {
+   const campusId = event.target.value
+   const select = this.element.querySelector('#id_events')

+   const response = await fetch(`/api/get-events/?campus_id=${campusId}`)
+   const { events } = await response.json()
+
+   select.innerHTML = ''
+   events.forEach(e => select.add(new Option(e.name, e.id)))
+   // First option is the top-priority event, so it ends up selected by default
+ }

            async handleCampusChange(event) {
                const campusId = event.target.value;
Lines 310–318
                loadingIndicator?.classList.remove('d-none');
                scheduleSelect.disabled = true;

+               /** Triggers the seletion of the Service Schedule
+                *
+                */
                try {
                    const response = await fetch(`/api/get-schedules/?campus_id=${campusId}`);
                    const data = await response.json();
```